

# ICS1513 - Computer Networks Laboratory

## Experiment 2: Multi-Client Chat Application using TCP & UDP Sockets

Nitin H - 3122247001042

July 15 2026

### Aim

To emulate a multi-client chat application between a server and multiple clients using the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP), where clients query a server-side student database (`students.csv`) by roll number and receive the corresponding student record.

### Question Summary

Develop client/server socket programs that emulate a multi-client chat-style lookup application:

- Implement the same request/response lookup service twice – once using TCP (connection-oriented, `SOCK_STREAM`) sockets, and once using UDP (connectionless, `SOCK_DGRAM`) sockets.
- The server should be able to service multiple clients querying by roll number, returning the matching student's details from a CSV-based database, or an appropriate "not found" message.
- Test both implementations with valid roll numbers, invalid/non-existent roll numbers, and empty/blank input, to verify correct request handling and response delivery in each case.

## 1 TCP Sockets

### Server Algorithm

1. Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`.
2. Set `SO_REUSEADDR` using `setsockopt()` to allow immediate re-binding.
3. Bind the socket to `INADDR_ANY` on port 8080 using `bind()`.
4. Put the socket into listening mode using `listen()` with a backlog of 5.
5. Loop forever:
  - (a) Accept an incoming client connection using `accept()`, obtaining a new socket descriptor.
  - (b) Spawn a new thread (`pthread_create`) running `handle_client()` to service the client concurrently, and detach it (`pthread_detach`).
6. Inside `handle_client()`:
  - (a) Read the roll number sent by the client using `read()`.
  - (b) Strip trailing newline/carriage-return characters.

- (c) Open `students.csv` and search line by line using `strtok()` on the comma delimiter, comparing the first field (roll number) with the received query.
- (d) If a match is found, construct a response string containing the roll number and student name; otherwise construct a "Not Found" message.
- (e) Send the response back to the client using `write()`.
- (f) Close the client socket.

## Client Algorithm

1. Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`.
2. Populate a `sockaddr_in` structure with the server's IP address (127.0.0.1) and port (8080) using `inet_pton()`.
3. Connect to the server using `connect()`.
4. Prompt the user for a roll number and read it from `stdin` using `fgets()`, stripping the trailing newline.
5. Send the roll number to the server using `write()`.
6. Read the server's response using `read()` and print it to the console.
7. Close the socket.

## Server Code (Q21Server.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <arpa/inet.h>
6  #include <pthread.h>
7
8  void *handle_client(void *client_void_ptr)
9  {
10     int client_fd = *(int *)client_void_ptr;
11     free(client_void_ptr);
12
13     char buffer[1024] = {0};
14     char response[1024] = {0};
15
16     ssize_t bytes_read = read(client_fd, buffer, sizeof(buffer) - 1);
17     if (bytes_read <= 0) {
18         close(client_fd);
19         return NULL;
20     }
21
22     buffer[strcspn(buffer, "\r\n")] = 0;
23
24     FILE *file = fopen("students.csv", "r");
25     if (file == NULL) {
26         perror("Failed to open students.csv");
27         strcpy(response, "Error: students.csv not found");
28         write(client_fd, response, strlen(response));
29         close(client_fd);
30         return NULL;
31     }
32
33     char linebuf[1024];
34     int found = 0;
35
36     while (fgets(linebuf, sizeof(linebuf), file) != NULL)
37     {

```

```

38     linebuf[strcspn(linebuf, "\r\n")] = 0;
39
40     char temp_line[1024];
41     strcpy(temp_line, linebuf);
42
43     char *roll_no = strtok(temp_line, ",");
44     if (roll_no && strcmp(roll_no, buffer) == 0)
45     {
46         char *name = strtok(NULL, ",");
47         if (name) {
48             snprintf(response, sizeof(response), "Roll_Number:_%s\nStudent_Name:_%s", roll_no, name);
49         } else {
50             snprintf(response, sizeof(response), "Roll_Number:_%s\nStudent_Name:_Unknown", roll_no);
51         }
52         found = 1;
53         break;
54     }
55 }
56
57 fclose(file);
58
59 if (!found) {
60     snprintf(response, sizeof(response), "Student_Record_Not_Found_for_roll_number_%s", buffer);
61 }
62
63 if (write(client_fd, response, strlen(response)) < 0) {
64     perror("Failed_to_send_response");
65 }
66
67 close(client_fd);
68 return NULL;
69 }
70
71 int main()
72 {
73     int server_fd;
74     struct sockaddr_in server_addr;
75
76     server_fd = socket(AF_INET, SOCK_STREAM, 0);
77     if (server_fd < 0) {
78         perror("Socket_creation_failed");
79         exit(EXIT_FAILURE);
80     }
81
82     int opt = 1;
83     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
84
85     server_addr.sin_family = AF_INET;
86     server_addr.sin_addr.s_addr = INADDR_ANY;
87     server_addr.sin_port = htons(8080);
88
89     if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
90         perror("Bind_failed");
91         close(server_fd);
92         exit(EXIT_FAILURE);
93     }
94
95     if (listen(server_fd, 5) < 0) {
96         perror("Listen_failed");
97         close(server_fd);
98         exit(EXIT_FAILURE);
99     }
100
101     printf("Server_is_waiting_on_port_8080...\n");
102
103     while (1)
104     {
105         int *new_sock = malloc(sizeof(int));

```

```

106     if (new_sock == NULL) {
107         perror("Malloc_failed");
108         continue;
109     }
110
111     *new_sock = accept(server_fd, NULL, NULL);
112     if (*new_sock < 0) {
113         perror("Accept_failed");
114         free(new_sock);
115         continue;
116     }
117
118     pthread_t thread_id;
119     if (pthread_create(&thread_id, NULL, handle_client, (void *)new_sock) == 0) {
120         pthread_detach(thread_id);
121     } else {
122         perror("Could_not_create_thread");
123         close(*new_sock);
124         free(new_sock);
125     }
126 }
127
128 close(server_fd);
129 return 0;
130 }

```

## Client Code (Q21Client.c)

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <unistd.h>
4  #include <arpa/inet.h>
5
6  int main() {
7      int sock;
8      struct sockaddr_in server_addr;
9      char msg[1024];
10     char response[1024];
11
12     sock = socket(AF_INET, SOCK_STREAM, 0);
13     if (sock < 0) {
14         perror("Socket_creation_failed");
15         return 1;
16     }
17
18     server_addr.sin_family = AF_INET;
19     server_addr.sin_port = htons(8080);
20
21     if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <= 0) {
22         perror("Invalid_server_address");
23         close(sock);
24         return 1;
25     }
26
27     if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
28         perror("Connection_failed");
29         close(sock);
30         return 1;
31     }
32
33     printf("Enter_a_roll_number:");
34     if (fgets(msg, sizeof(msg), stdin) == NULL) {
35         close(sock);
36         return 0;
37     }
38     msg[strcspn(msg, "\n")] = 0;

```

```

39
40     if (write(sock, msg, strlen(msg)) < 0) {
41         perror("Failed to send request");
42         close(sock);
43         return 1;
44     }
45
46     ssize_t bytes_read = read(sock, response, sizeof(response) - 1);
47     if (bytes_read > 0) {
48         response[bytes_read] = '\0';
49         printf("\nServer response: %s\n", response);
50     } else {
51         perror("Failed to receive response");
52     }
53
54     close(sock);
55     return 0;
56 }

```

## 2 UDP Sockets

### Server Algorithm

1. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. Populate a `sockaddr_in` structure with `INADDR_ANY` and port 8080.
3. Bind the socket to this address using `bind()`. (Note: UDP requires no `listen()` or `accept()` call, since it is connectionless.)
4. Loop forever:
  - (a) Clear the buffer and response arrays.
  - (b) Receive a datagram (the roll number query) from a client using `recvfrom()`, which also captures the client's address.
  - (c) Call `search_student()` to look up the roll number in `students.csv` using the same `strtok()`-based parsing as the TCP version.
  - (d) Send the resulting response back to the querying client's address using `sendto()`.

### Client Algorithm

1. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. Populate a `sockaddr_in` structure with the server's IP (127.0.0.1) and port (8080) using `inet_addr()`.
3. Prompt the user for a roll number and read it using `fgets()`, stripping the newline.
4. Send the roll number directly to the server's address using `sendto()` (no prior connection required).
5. Wait to receive the response datagram using `recvfrom()`.
6. Print the received response to the console.
7. Close the socket.

## Server Code (Q22Server.c)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define BUFFER_SIZE 1024
9  #define CSV_FILE "students.csv"
10
11
12 void search_student(const char *roll_no, char *response) {
13     FILE *file = fopen(CSV_FILE, "r");
14     if (!file) {
15         strcpy(response, "Error: Server database missing.");
16         return;
17     }
18
19     char line[BUFFER_SIZE];
20     int found = 0;
21
22     while (fgets(line, sizeof(line), file)) {
23
24         line[strcspn(line, "\n")] = 0;
25
26
27         char temp_line[BUFFER_SIZE];
28         strcpy(temp_line, line);
29
30         char *token = strtok(temp_line, ",");
31         if (token && strcmp(token, roll_no) == 0) {
32
33             char *name = strtok(NULL, ",");
34             if (name) {
35                 snprintf(response, BUFFER_SIZE, "Roll Number: %s\nStudent Name: %s", token, name);
36             } else {
37                 snprintf(response, BUFFER_SIZE, "Roll Number: %s\nStudent Name: Unknown", token);
38             }
39             found = 1;
40             break;
41         }
42     }
43
44     fclose(file);
45
46     if (!found) {
47         strcpy(response, "Student Record Not Found");
48     }
49 }
50
51 int main() {
52     int sockfd;
53     char buffer[BUFFER_SIZE];
54     char response[BUFFER_SIZE];
55     struct sockaddr_in server_addr, client_addr;
56     socklen_t addr_len = sizeof(client_addr);
57
58
59     if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
60         perror("Socket creation failed");
61         exit(EXIT_FAILURE);
62     }
63
64     memset(&server_addr, 0, sizeof(server_addr));
65     server_addr.sin_family = AF_INET;
66     server_addr.sin_addr.s_addr = INADDR_ANY;
```

```

67     server_addr.sin_port = htons(PORT);
68
69
70     if (bind(sockfd, (const struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
71         perror("Bind failed");
72         close(sockfd);
73         exit(EXIT_FAILURE);
74     }
75
76     printf("UDP Server is running on port %d...\n", PORT);
77
78     while (1) {
79         memset(buffer, 0, BUFFER_SIZE);
80         memset(response, 0, BUFFER_SIZE);
81
82
83         int n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0, (struct sockaddr *)&client_addr, &addr_len);
84         if (n < 0) continue;
85
86         buffer[n] = '\0';
87         printf("Received request for Roll Number: %s\n", buffer);
88
89
90         search_student(buffer, response);
91
92
93         sendto(sockfd, response, strlen(response), 0, (const struct sockaddr *)&client_addr, addr_len);
94         printf("Response sent back to client.\n\n");
95     }
96
97     close(sockfd);
98     return 0;
99 }

```

## Client Code (Q22Client.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define BUFFER_SIZE 1024
9
10 int main() {
11     int sockfd;
12     char buffer[BUFFER_SIZE];
13     char roll_no[BUFFER_SIZE];
14     struct sockaddr_in server_addr;
15
16
17     if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
18         perror("Socket creation failed");
19         exit(EXIT_FAILURE);
20     }
21
22     memset(&server_addr, 0, sizeof(server_addr));
23     server_addr.sin_family = AF_INET;
24     server_addr.sin_port = htons(PORT);
25     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
26
27
28     printf("Enter Roll Number to search: ");
29     if (fgets(roll_no, sizeof(roll_no), stdin) == NULL) {
30         close(sockfd);

```

```

31     return 0;
32 }
33
34 roll_no[strcspn(roll_no, "\n")] = 0;
35
36
37 sendto(sockfd, roll_no, strlen(roll_no), 0, (const struct sockaddr *)&server_addr, sizeof(server_addr))
38     ;
39
40 socklen_t addr_len = sizeof(server_addr);
41 int n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0, (struct sockaddr *)&server_addr, &addr_len);
42
43 if (n >= 0) {
44     buffer[n] = '\0';
45     printf("\n---Server_Response---\n%s\n-----\n", buffer);
46 } else {
47     perror("Failed to receive data");
48 }
49
50
51 close(sockfd);
52 return 0;
53 }

```

## Sample Input and Output

### TCP

**Input:** Roll numbers 101, 103, 111 entered across separate client runs.

#### Output:

```

Enter a roll number: 101
Server response:
Roll Number: 101
Student Name: Alice Smith

```

```

Enter a roll number: 103
Server response:
Roll Number: 103
Student Name: Charlie Brown

```

```

Enter a roll number: 111
Server response:
Student Record Not Found for roll number 111

```

```
CHAT  TERMINAL  CLAUDE CODE  wsl + v
(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory]
$ cd Experiment\ 2
(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ gcc Q21Server.c -o c
(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ gcc Q21Server.c -o s
(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./s
Server is waiting on port 8080...

L$ ./c
Enter a roll number: 101

Server response:
Roll Number: 101
Student Name: Alice Smith

(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./c
Enter a roll number: 103

Server response:
Roll Number: 103
Student Name: Charlie Brown

(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./c
Enter a roll number: 111

Server response:
Student Record Not Found for roll number 111

(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$
```

Figure 1: TCP server and client output – valid roll numbers (101, 103) and a not-found case (111)

## UDP

**Input:** Roll numbers 101, 102 (with a stray leading backslash), a blank entry, and 111 entered across separate client runs.

**Output:**

```
Enter Roll Number to search: 101
--- Server Response ---
Roll Number: 101
Student Name: Alice Smith
```

```
Enter Roll Number to search: 102
--- Server Response ---
Student Record Not Found
```

```
Enter Roll Number to search:
--- Server Response ---
Student Record Not Found
```

```
Enter Roll Number to search: 111
--- Server Response ---
(Response sent back to client)
```

| TCP                     | UDP                             |
|-------------------------|---------------------------------|
| SOCK_STREAM             | SOCK_DGRAM                      |
| bind() + listen()       | Only bind()                     |
| accept()                | No accept()                     |
| read(), write(), send() | recvfrom(), sendto()            |
| 3-Way Handshake         | No handshake                    |
| Guaranteed Delivery     | No guarantee                    |
| Single Stream, Unicast  | No recovery, supports Multicast |

Table 1: TCP vs UDP comparison

## Learning Outcomes

- Learnt to implement TCP sockets between a server and multiple clients using a multi-threaded, connection-oriented approach.
- Learnt to implement UDP sockets between a server and multiple clients using a connectionless, datagram-based approach.
- Learnt to implement socket programming system calls such as `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()`, and `sendto()/recvfrom()`.
- Understood the key differences between TCP and UDP in terms of connection setup, reliability, and delivery guarantees.

```
CHAT  TERMINAL  CLAUDE CODE

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./s
UDP Server is running on port 8080...
Received request for Roll Number: 101
Response sent back to client.

Received request for Roll Number: \102
Response sent back to client.

Received request for Roll Number:
Response sent back to client.

Received request for Roll Number: 111
Response sent back to client.

[]

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./c
Enter Roll Number to search: 101

--- Server Response ---
Roll Number: 101
Student Name: Alice Smith
-----

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./c
\Enter Roll Number to search: 102

--- Server Response ---
Student Record Not Found
-----

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 2]
$ ./c
Enter Roll Number to search:

--- Server Response ---
Student Record Not Found
```

Figure 2: UDP server and client output – valid, malformed, blank, and additional roll-number queries